In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn import tree
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, mean_squared_error, confusion_matrix, c
```

In [3]:
```
df = pd.read_csv('loan_preprocessed.csv')
```

In [4]:
```
df = df.drop("Loan_ID", axis=1)
```

In [6]:
```
df.head()
```

Out[6]:

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	Coapplicant
0	Male	No	0	Graduate	No	0.164300	-0
1	Male	Yes	1	Graduate	No	-0.125902	0
2	Male	Yes	0	Graduate	Yes	-0.488770	-0
3	Male	Yes	0	Not Graduate	No	-0.584358	0
4	Male	No	0	Graduate	No	0.198914	-0

In [56]:
```
df.describe()
```

Out[56]:

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_His
count	6.070000e+02	6.070000e+02	6.070000e+02	6.070000e+02	607.000
mean	-1.024258e-16	-7.316132e-17	7.901423e-17	-1.697343e-16	0.855
std	1.000825e+00	1.000825e+00	1.000825e+00	1.000825e+00	0.352
min	-1.142069e+00	-8.180313e-01	-1.699587e+00	-5.167827e+00	0.000
25%	-5.136409e-01	-8.180313e-01	-5.504616e-01	2.670672e-01	1.000
50%	-3.024074e-01	-1.595007e-01	-1.968847e-01	2.670672e-01	1.000
75%	1.445869e-01	4.598115e-01	2.324587e-01	2.670672e-01	1.000
max	7.992432e+00	5.558486e+00	6.394800e+00	2.141169e+00	1.000

In [57]:
```
df = df.drop("Loan_ID", axis=1)
```

In [58]:
```
le = LabelEncoder()

for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
```

In [59]:
```
X = df.drop("Loan_Status", axis=1)
y = df["Loan_Status"]
```

In [60]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

In [61]:
```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

SVM Implementation

In [62]:
```
svm_model = SVC(kernel='linear')
svm_model.fit(X_train_scaled, y_train)
```

Out[62]:

▾ SVC ⓘ ?

▸ Parameters

In [ ]:
```
```

In [63]:
```
y_pred_linear = svm_model.predict(X_test_scaled)
linear_accuracy = accuracy_score(y_test, y_pred_linear)
```

In [64]:
```
print("Linear SVM Accuracy:", linear_accuracy)
```

```
Linear SVM Accuracy: 0.8114754098360656
```

In [83]:
```
print(confusion_matrix(y_test, y_pred_linear))
```

```
[[14 22]
 [ 1 85]]
```

In [84]:
```
print(classification_report(y_test, y_pred_linear))
```

```
              precision    recall  f1-score   support

           0       0.93      0.39      0.55        36
           1       0.79      0.99      0.88        86

    accuracy                           0.81       122
   macro avg       0.86      0.69      0.71       122
weighted avg       0.84      0.81      0.78       122
```

Comparing with Polynomial

In [65]:
```
svm_poly = SVC(kernel='poly', degree=3)
svm_poly.fit(X_train_scaled, y_train)
```

Out[65]:

▾ SVC ⓘ ?

▸ Parameters

In [66]:
```
y_pred_poly = svm_poly.predict(X_test_scaled)
poly_accuracy = accuracy_score(y_test, y_pred_poly)
```

In [67]:
```
print("Polynomial SVM Accuracy:", poly_accuracy)
```

```
Polynomial SVM Accuracy: 0.7868852459016393
```

Decision Tree Implementation

In [75]:
```
dt_model = DecisionTreeClassifier(random_state=42, max_depth=5)
```

In [76]:
```
dt_model.fit(X_train, y_train)
```

Out[76]:

▾ DecisionTreeClassifier ⓘ ?

▸ Parameters

In [77]:
```
y_pred_dt = dt_model.predict(X_test)
```

In [78]:
```
dt_accuracy = accuracy_score(y_test, y_pred_dt)
print("Decision Tree Accuracy:", dt_accuracy)
```

```
Decision Tree Accuracy: 0.7786885245901639
```

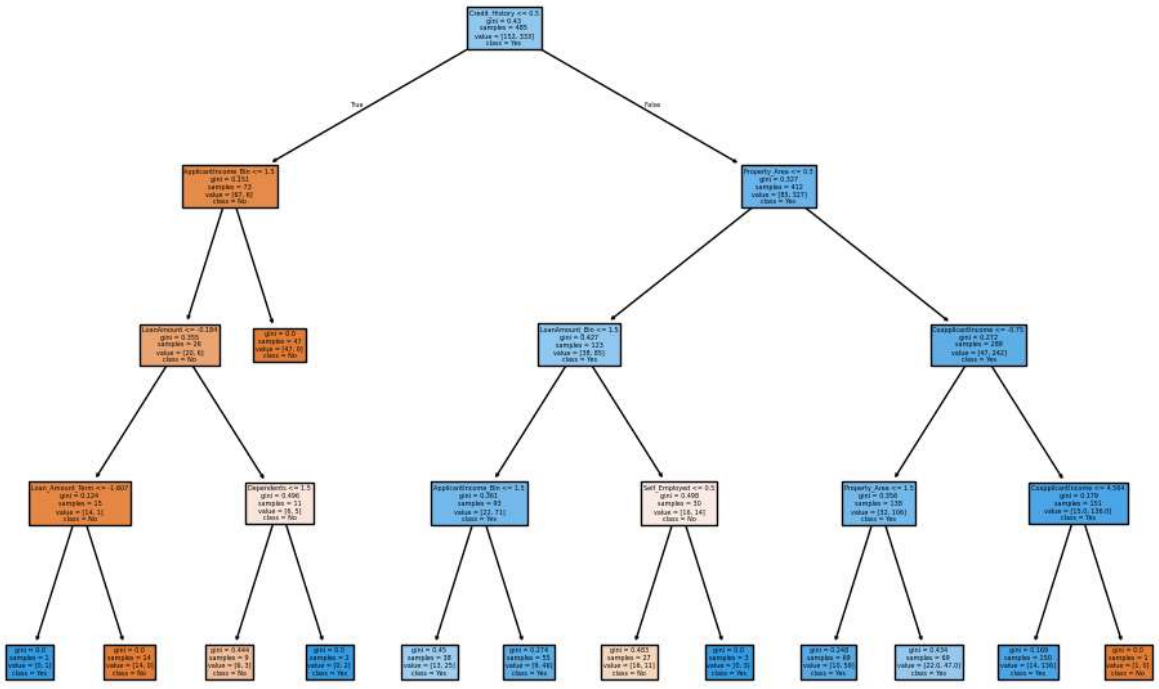
In [81]:
```
print(confusion_matrix(y_test, y_pred_dt))
```

```
[[12 24]
 [ 3 83]]
```

In [82]:
```
print(classification_report(y_test, y_pred_dt))
```

```
              precision    recall  f1-score   support

           0       0.80      0.33      0.47        36
           1       0.78      0.97      0.86        86

    accuracy                           0.78       122
   macro avg       0.79      0.65      0.67       122
weighted avg       0.78      0.78      0.75       122
```

In [74]:
```
plt.figure(figsize=(12,8))

tree.plot_tree(
    dt_model,
    feature_names=X.columns,
    class_names=['No','Yes'],
    filled=True
)

plt.show()
```



Comparing Accuracy

In [85]:
```
print("\nModel Accuracy Comparison")
print("Linear SVM:", linear_accuracy)
print("Polynomial SVM:", poly_accuracy)
print("Decision Tree:", dt_accuracy)
```

```
Model Accuracy Comparison
Linear SVM: 0.8114754098360656
Polynomial SVM: 0.7868852459016393
Decision Tree: 0.7786885245901639
```

In [86]:
```
models = ['Linear SVM', 'Polynomial SVM', 'Decision Tree']
accuracies = [linear_accuracy, poly_accuracy, dt_accuracy]

plt.figure(figsize=(6,4))
plt.bar(models, accuracies)
```

```
plt.title("Model Accuracy Comparison")
plt.xlabel("Models")
plt.ylabel("Accuracy")

plt.show()
```